

CS4262/5462 Machine Learning Systems

Project 1: LLM Serving

Last Updated: Mar 9, 2026

This document may be updated. Please revisit it regularly for the latest requirements and instructions.

- **09 March.** Clarification of Track B’s data format: The mention of “multi-turn dialogue” in the background section of Track B is only to provide real-world motivation for the task design. For this track, you only need to handle single-turn dialogues conforming to the format of our released data. The validation and test data follow the same distribution.

Instructions

- **Collaboration Policy:** Projects may be completed individually or in groups of up to 4 students. You must include a “Contribution Statement” in the appendix of your final report, detailing the specific workload and percentage contribution of each member.
- **Online Submission:** All deliverables (code, report, and demo video) must be submitted via the course Canvas site. Please ensure all files are compressed into a single ZIP file unless otherwise instructed.
- **Generative AI Policy:** You are permitted to use Generative AI tools (e.g., ChatGPT, Copilot) for this project. However, you are required to document your usage: specify which tools were used and include the prompts/queries in your report. **Note:** You are fully responsible for the correctness and compilation of your code; “AI hallucination” is not a valid excuse for errors.
- **Late Policy:** Please strictly follow the deadlines. Late submissions will be penalized at a rate of [10%] per day. Submissions delayed by more than 3 days will not be accepted.
- **Vast.ai Policy:** Each team can claim USD \$8, which covers more than 60 hours of one 5080 GPU. Submit the receipt to the TA to get reimbursement.

In this project, you will build and optimize an containerized LLM serving engine for one of two given scenarios: dynamic agentic workflows or customer support chatbot. Both scenarios are highly useful in production scenarios nowadays.

1 Overview of Two Tracks

This sections go through the motivations, goals, and deliverables for each of the two tracks.

1.1 Track A. Dynamic Workflow Serving with Control Flows

Background. We are currently witnessing a paradigm shift in how Large Language Models (LLMs) are deployed. While the first wave of LLM applications focused on stateless “Chatbot” interactions, the frontier has moved toward *agentic Workflows*. In these dynamic workflows, the LLM is no longer just a text generator; it is the “CPU” of a complex system. It makes decisions, calls tools, loops until a condition is met, and branches logic based on its own outputs. Examples include:

- ReAct Agents: Reason → Act → Observe → Loop.

- Coding Assistants: Write Code → Run Tests → Fix Errors → Repeat.
- Tree of Thoughts: Explore multiple reasoning paths and backtrack if necessary.

Goals. Current serving infrastructure is often optimized for the “Chatbot” paradigm. Agentic workflows are typically executed by treating LLM inference as an external blackbox API, missing a lot of optimization opportunities. In this track, you will build a serving engine for LLM-based agentic workflows. Instead of treating the LLM as a black box API, your engine will accept the entire workflow graph and execute it directly on the server, enabling more holistic optimizations, such as prefetching and workflow-aware KV cache eviction policy.

Deliverables. A Docker image that serves your LLM engine and exposes the following HTTP endpoints:

- GET /health: liveness check;
- GET /ready: engine readiness check;
- POST /v1/workflow: submit a workflow to execute. Returns a trace for each step, containing the output and log probabilities.

1.2 Track B. Customer Support Chatbot.

Background. In customer service applications, the primary challenge is not complex control flow, but massive concurrency and strict Service Level Agreements (SLAs). Here, the LLM acts as a massively parallel worker, maintaining context for thousands of simultaneous user sessions. Each session consists of multi-turn dialogue where low latency (Time-To-First-Token) is critical for user satisfaction. Examples include:

- E-commerce Support: Handling thousands of product queries simultaneously during peak hours.
- Automated Triage: Classifying and responding to support tickets in real-time.
- Interactive Troubleshooting: Guiding users through multi-step diagnostic processes with instant feedback.

Goals. In this track, you will build a high-throughput serving engine optimized for interactive chat, with short context lengths and high concurrency. Also, some requests might be repeated after a long time, so cache eviction and recomputation strategy needs to be carefully designed.

Deliverables. A Docker image that serves your LLM engine and exposes the following HTTP endpoints:

- GET /health: liveness check;
- GET /ready: engine readiness check;
- POST /v1/chat/completions: submit a chat history and generate a response, containing the output text and log probabilities.

2 Evaluation

To benchmark the performance, we have prepared 3k workflows and 40k customer service inquiries, split into three equal-size portions:

1. The training split: Distributed along with the benchmark suite for local testing.
2. The validation split: Kept confidential for leaderboard performance reporting.
3. The test split: Kept confidential for grading.

The requests will be issued at a concurrency of 16 for track A and 128 for track B. The performance is evaluated in the following dimensions:

- End-to-end P50 and P95 latency.
- Throughput, i.e., workflows per second for track A and requests per second for track B.
- Perplexity of all outputs.

For track A, we also measure the average trace length. Perplexity and the trace lengths are used to measure the LLM quality degradation.

3 Starter Kit and Deployment Instructions

We have prepared a starter kit for both tracks to help you get started. It contains

1. A functional codebase that implements all required APIs using vLLM.
2. A benchmark client for evaluating the performance.
3. The public data for local testing.

The codebase is in https://github.com/mlsys-io/mlsys_llm_framework, and the benchmark scripts plus public data are available at https://github.com/mlsys-io/mlsys_llm_benchmark.

Feel free to modify any part of the codebase as long as the final engine preserves the expected HTTP endpoint for benchmarking.

3.1 Local deployment

We have provided a `Dockerfile` and a `docker-compose.yaml` in the codebase. To deploy the engine locally, run `docker compose up --build -d`.

Note that if you are running on a non-AMD64 machine (e.g. ARM), you need to comment out the `platform: linux/amd64` line in the Docker Compose Config so it builds the image targeting your platform.

3.2 Deployment with Vast AI

We will provide each group that selects this project with some Vast AI credits, so that you can test your containerized engine on an NVIDIA GeForce RTX 5080.

Below we provide a step-by-step instructions to deploy the engine on Vast AI. The following instructions take the first track as an example. If you choose the second track, replace all `agent-engine` below with `chat-engine`.

1. Create a GitHub Personal Access Token in <https://github.com/settings/tokens/> with `write:packages` and `read:packages` permissions.
2. Connect your docker to the GitHub registry: `echo $YOUR_GITHUB_PAT | docker login ghcr.io -u $GITHUB_USERNAME --password-stdin`.
3. Build and push the image:

```
1 docker compose build
2 docker tag agent-engine:latest ghcr.io/$GITHUB_USERNAME/agent-engine:latest
3 docker push ghcr.io/$GITHUB_USERNAME/agent-engine:latest
```

Please make sure you have chosen the correct platform if you are building the image on a non-AMD64 platform. After you push the image, please take note of the SHA256 digest of the pushed image. You may also check the digest by

```
1 docker inspect --format='{{index .RepoDigests 0}}' ghcr.io/$GITHUB_USERNAME/agent-engine:latest
```

The digest is necessary because it is an immutable signature for an image.

4. Log in to Vast.ai and go to the Templates page <https://cloud.vast.ai/templates/>. Click “Create New Template”, set the relevant fields as following:
 - Image Path: `ghcr.io/YOUR_GITHUB_USERNAME/agent-engine@sha256:YOUR_DIGEST`;
 - Docker Options: Add `-p 8000:8000` to expose the port.
 - Launch Mode: “Docker ENTRYPOINT”.
 - Docker Repository Authentication: Set Server as `ghcr.io` and input your github username and PAT accordingly.
 - Disk Space: We suggest at least 128GB as the image and weights already take around 40GB.
5. At <https://cloud.vast.ai/create/>, select your custom template, and rent an “1x RTX 5080” instance. As the custom image is not cached in the instance, the loading time depends on the network download speed. On an instance with around 2Gbps download speed, it takes around 30 minutes to set up the instance. Therefore, please be patient in waiting for the engine to be ready.
6. After the instance status becomes “Running”, open the “IP & Port Info” tab of the instance by clicking on the IP address. You can find at the bottom the external port `$VAST_PORT` mapping to the internal 8000 port. You could access the engine at `http://$VAST_IP:$VAST_PORT`.

Note that Flash Attention has some known issues (<https://github.com/Dao-AILab/flash-attention/issues/2168>), which causes problems when trying to deploy the engine on Vast AI instances with Max CUDA 12.8. In our codebase, we set the attention backend to FlashInfer in the Dockerfile so that it can work with any Vast AI instances with Max CUDA not less than 12.8. If you want to use Flash attention anyway, please use Vast AI instances with Max CUDA at least 13.0.

4 Submission and Grading

To ensure your submission is graded correctly, you must submit the following items via Canvas.

4.1 How to Submit

1. Project report: A PDF report that summarizes the optimizations adopted to optimize the engine.
2. Source code: Archive your source code into a zip file and submit.
3. The full image reference with digest: To ensure the image is pushed before the deadline, you should submit the image URI with the immutable **SHA256 digest**. The full string will be in the form of `ghcr.io/username/project@sha256:a1b2c3d4....`.
4. Read-only access token with the associated GitHub user name: A personal access token (PAT) with `read:packages` permission so we can pull the image from your private GitHub registry.

Note that our grading server runs on x86_64 (amd64) architecture. Please verify that your image is built for the amd64 platform before submission:

```
1 docker image inspect ghcr.io/user/project:tag --format '{{.Architecture}}'
```

4.2 How to grade

The grading will be based on multiple factors, including the novelty of the optimizations, the effect of optimizations, etc. This section elaborates on the performance benchmark, which is also an important dimension of the final grade.

The evaluation will be carried out on a single NVIDIA RTX 5080, using the benchmark suite included in the starter kit. Outbound network connection will be disabled to ensure that the engine does not use external resources (e.g. API calling). The huggingface cache containing the model will be provided in the volume mounted at `/root/.cache/huggingface`.

We will host a leaderboard that is updated weekly for you to test your engine. We will check the correctness using the private correctness data split and benchmark the performance using the validation benchmark data split.